# Digital Design and Computer Architecture

Laurin Seeholzer

March 29, 2026

# Contents


**1 Fundamentals** **3**
1.1 Metal-Oxide-Semiconductor Transistors . . . 3
1.2 Logic Gates . . . 3
1.3 Boolean Algebra . . . 4

**2 Combinational Logic** **6**
2.1 Combinational Building Blocks . . . 6

**3 Sequential Logic** **7**
3.1 Fundamental Building Blocks . . . 7
3.2 Timing and the Clock . . . 7
3.3 Finite State Machines (FSM) . . . 8
3.4 Memory and Registers . . . 9

**4 Timing** **10**
4.1 Combinational Circuit Timing . . . 10
4.2 Sequential Circuit Timing . . . 10
4.3 Clock Skew and Performance . . . 11

**5 Von Neumann Model and ISA** **12**
5.1 Instruction Set Architecture (ISA) . . . 13
5.2 Assembly . . . 14

**6 Microarchitecture and Processor Design** **15**
6.1 Fundamentals . . . 15
6.2 Single-Cycle Microarchitecture . . . 15
6.3 Multi-Cycle Microarchitecture . . . 16

**7 Pipelining** **18**
7.1 Principles of Pipelining . . . 18
7.2 The 5-Stage Instruction Pipeline . . . 18
7.3 Pipeline Hazards and Dependencies . . . 19
7.4 Resolving Data Hazards . . . 19
7.5 Control Hazards and Branching . . . 20
7.6 Advanced Pipeline Issues . . . 20

# 1 Fundamentals

## 1.1 Metal-Oxide-Semiconductor Transistors

**Definition 1.1.** *An **n-type** MOS transistor acts as a switch that is **closed (ON)** when the Gate voltage is High (1) and **open (OFF)** when the Gate voltage is Low (0). It is efficient at passing a logic 0 (strong 0) but inefficient at passing a logic 1 (weak 1).*

**Definition 1.2.** *A **p-type** MOS transistor is the dual of nMOS. It is **closed (ON)** when the Gate voltage is Low (0) and **open (OFF)** when the Gate voltage is High (1). It is efficient at passing a logic 1 (strong 1) but inefficient at passing a logic 0 (weak 0).*

**Definition 1.3.** *In **Complementary MOS (CMOS)** logic, transistors are organized into two networks:*

- ***Pull-Up Network (PUN):*** *Built with pMOS to connect the output to $V_{DD}$ (1).*
- ***Pull-Down Network (PDN):*** *Built with nMOS to connect the output to Ground (0).*

*This ensures the output is always driven strongly to either 1 or 0 without drawing significant static power.*

**Example 1.4.** *The simplest CMOS gate is the NOT gate. It consists of one pMOS in the PUN and one nMOS in the PDN, both connected to the same input A.*

$$Y = \overline{A}$$

## 1.2 Logic Gates

**Definition 1.5.** ***NOT:*** *The simplest gate; it flips the input.*

- ***Logic:*** $Y = \overline{A}$
- ***Transistors:*** *2 (1 pMOS, 1 nMOS)*

**Definition 1.6.** ***NAND:*** *Outputs 0 only if both inputs are 1.*

- ***Logic:*** $Y = \overline{A \cdot B}$
- ***Transistors:*** *4 (2 pMOS parallel, 2 nMOS series)*

**Definition 1.7.** ***NOR:*** *Outputs 1 only if both inputs are 0.*

- ***Logic:*** $Y = \overline{A + B}$
- ***Transistors:*** *4 (2 pMOS series, 2 nMOS parallel)*

**Remark 1.8.** ***AND/OR*** *are typically implemented as NAND/NOR followed by an inverter because inverting gates are more natural in CMOS. (**NAND/NOR** are universally used because they are efficient to build with 4 transistors for 2 inputs.)*

**Remark 1.9.** ***XOR/XNOR:*** *Used for comparisons and parity.*

- ***XOR:*** $Y = A \oplus B$ *(1 if inputs are **different**)*
- ***XNOR:*** $Y = \overline{A \oplus B}$ *(1 if inputs are the **same**)*

**Definition 1.10.** ***Buffer:*** *The output equals the input ($Y = A$). It is used for signal amplification or delay and is physically implemented as two NOT gates in series.*

## 1.3 Boolean Algebra

**Definition 1.11.** *Boolean algebra (and therfore logic gates) follow specific mathematical rules:*

- ***Identity:*** $A \cdot 1 = A$ *and* $A + 0 = A$
- ***Complement:*** $A \cdot \overline{A} = 0$ *and* $A + \overline{A} = 1$
- ***Distributive:*** $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$

**Theorem 1.12.** ***DeMorgan's theorems*** *allow for the conversion between AND/OR and inverted logic:*

$$\textit{Law 1: } \overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\textit{Law 2: } \overline{A + B} = \overline{A} \cdot \overline{B}$$

**Important 1.13.** *A set of gates is **logically complete** if any Boolean function can be realized using only those gates. The most common complete sets are $\{AND, OR, NOT\}$, $\{NAND\}$, and $\{NOR\}$.*

**Definition 1.14.** ***Minterm****: A product (AND) term containing all input variables in either true or complemented form.* ***Sum of Products (SOP)****: A function expressed as the OR of minterms for which the output is 1.*

**Algorithm 1.15**

```
To simplify logic using a Karnaugh Map (K-Map)

1. Map the truth table values into a grid where adjacent cells
   differ by only one bit (Gray code).
2. Circle groups of 2^n adjacent 1s (rectangular blocks).
3. Each circle represents a simplified product term where variables
    that change within the circle are eliminated.
```

# 2 Combinational Logic

**Definition 2.1.** *Combinational Logic is a circuit where the output is a pure function of the current inputs. It has no memory of past states.*

## 2.1 Combinational Building Blocks

**Definition 2.2.** *An $n$-to-$2^n$* ***decoder*** *interprets an $n$-bit binary input and sets exactly one of its $2^n$ outputs to $1$. It is primarily used for memory addressing and instruction decoding.*

**Definition 2.3.** *A* ***Multiplexer*** *uses select lines ($S$) to choose which data input ($D$) is routed to the single output ($Y$). For $N$ inputs, $log_2 N$ select lines are required.*

**Example 2.4.** *A MUX can implement any logic function by treating its inputs as a* ***Lookup Table (LUT)****. By wiring the data inputs to constants ($0$ or $1$) and using variables as select signals, the MUX reproduces the truth table.*

**Definition 2.5.** *A* ***Full Adder*** *is the building block of arithmetic units. It takes inputs $A, B$, and a carry-in $C_{in}$, producing:*

$$\textit{Sum: } S = A \oplus B \oplus C_{in}$$

$$\textit{Carry-out: } C_{out} = (A \cdot B) + (C_{in} \cdot (A \oplus B))$$

**Important 2.6.** *An $n$-bit* ***Ripple Carry Adder (RCA)*** *is constructed by chaining $n$ Full Adders. The $C_{out}$ of one stage becomes the $C_{in}$ of the next. The critical path (speed bottleneck) is the "ripple" effect as the carry bit travels from the LSB to the MSB.*

**Lemma 2.7.** *A* ***Tri-state buffer*** *has three possible output states: $0, 1$, and* ***High-Z*** *(High Impedance). When the enable signal is $0$, the buffer is electrically disconnected. This allows multiple components to share a single* ***Bus*** *wire.*

**Example 2.8.** *A* ***Programmable Logic Array (PLA)*** *is a structured logic device consisting of a programmable* ***AND-plane*** *(creating minterms) and a programmable* ***OR-plane*** *(summing the minterms). It is a physical implementation of the Sum-of-Products (SOP) form.*

# 3 Sequential Logic

## 3.1 Fundamental Building Blocks

**Definition 3.1.** *Unlike combinational logic, where the output depends solely on current inputs, sequential logic produces outputs based on both current and past input values. These circuits incorporate storage elements to maintain a "state".*

**Definition 3.2.** *The most basic storage element consists of two inverters connected in a feedback loop. It has two stable states ($Q = 1$ or $Q = 0$) and a potential metastable state where outputs oscillate.*

**Definition 3.3.** *A Reset-Set (R-S) latch is typically built from cross-coupled NAND gates.*

- ***Set*** *($S = 0$):* *Forces the output $Q$ to $1$.*
- ***Reset*** *($R = 0$):* *Forces the output $Q$ to $0$.*
- ***Forbidden*** *($S = 0, R = 0$):* *This state is invalid as it breaks the $Q = \overline{Q'}$ invariant and can lead to metastability.*

**Definition 3.4.** *The Gated D Latch adds a Write Enable ($WE$) signal to an R-S latch to control when data is stored. When $WE = 1$, the output $Q$ follows the data input $D$; otherwise, it holds the previous value ($Q_{prev}$).*

**Important 3.5.** *Latches are "transparent" while their enable signal is high, meaning any change in $D$ immediately propagates to $Q$. This is undesirable for state registers in synchronous systems because next-state values could overwrite current-state values before the cycle ends.*

**Definition 3.6.** *A D Flip-Flop solves the transparency problem by using a master-slave configuration of two D latches. It is* ***edge-triggered****, capturing the value of $D$ only at the rising edge of the clock ($0 \to 1$ transition) and maintaining it for the entire cycle.*

## 3.2 Timing and the Clock

**Definition 3.7.**

- ***Asynchronous:*** *State transitions occur immediately in response to inputs with no global coordination.*
- ***Synchronous:*** *State transitions are synchronized to a global* **clock** *signal and occur at fixed units of time.*

**Definition 3.8.** ***Clock (CLK):*** *A periodic signal alternating between* 0 *and* 1 *that triggers state transitions across all sequential elements in a system. The clock period must be long enough to accommodate the worst-case delay of the combinational logic.*

**Important 3.9.** *Standard synchronous registers use* ***rising-clock-edge*** *triggered flip-flops. This ensures that the state changes only once per clock cycle, making system behavior predictable and easier to design than level-triggered latch-based systems.*

## 3.3 Finite State Machines (FSM)

**Definition 3.10.** ***Finite State Machine (FSM):*** *A discrete-time model of a stateful system consisting of a finite number of states, external inputs, external outputs, state transitions, and output logic.*

**Theorem 3.11.** *FSMs are categorized by how they generate outputs:*

- ***Moore FSM:*** *Outputs depend* ***only*** *on the current state.*
- ***Mealy FSM:*** *Outputs depend on* ***both*** *the current state and the current inputs.*

**Remark 3.12.** *Every FSM implementation consists of three parts:*

1. ***State Register:*** *Sequential circuit storing the current state.*
2. ***Next State Logic:*** *Combinational circuit determining the next state based on current state and inputs.*
3. ***Output Logic:*** *Combinational circuit generating system outputs.*

**Example 3.13.** *States must be represented as binary values. Common methods include:*

- ***Binary Encoding:*** *Uses the minimum number of bits ($\log_2(\text{states})$); minimizes flip-flops but may lead to complex combinational logic.*
- ***One-Hot Encoding:*** *Uses one bit per state; simplifies next-state logic but maximizes flip-flop usage.*
- ***Output Encoding:*** *Uses the output signals themselves as the state bits; minimizes output logic.*

## 3.4 Memory and Registers

**Definition 3.14.** ***Register:*** *A structure that stores multiple bits (e.g., 4 or 32 bits) by grouping D flip-flops together with a common clock or write-enable signal.*

**Definition 3.15.** ***Memory:*** *An organization of storage locations indexed by addresses.*

- ***Address Space:*** *The total number of unique locations ($2^n$ for $n$ address bits).*
- ***Addressability:*** *The number of bits stored at each location.*
- ***Hardware:*** *Uses an **Address Decoder** to select a location and a **Multiplexer** to route the selected data to the output.*

# 4 Timing

## 4.1 Combinational Circuit Timing

**Definition 4.1.** *The timing behavior of a combinational circuit is bounded by two parameters:*

- ***Propagation Delay ($t_{pd}$):*** *The* ***maximum*** *time from when an input changes until the output has reached its final value (longest path).*
- ***Contamination Delay ($t_{cd}$):*** *The* ***minimum*** *time from when an input changes until the output* ***starts*** *to change (shortest path).*

> **Remark 4.2.** *Transistors take finite time to switch due to capacitance, resistance, and the finite speed of light. Actual delays vary based on temperature, supply voltage, circuit aging, and whether the signal is rising or falling.*

**Example 4.3.** *A* ***glitch*** *occurs when a single input transition causes multiple transitions on the output before it settles. This is caused by different paths through the logic having different delays. While often ignored in steady-state logic, glitches can be fixed by adding redundant "consensus terms" in a K-map to cover transitions between prime implicants.*

## 4.2 Sequential Circuit Timing

**Definition 4.4.** *For a flip-flop to reliably sample data at the rising clock edge, the input D must be stable within the* ***aperture time*** *($t_a = t_{setup} + t_{hold}$):*

- ***Setup Time ($t_{setup}$):*** *The time* ***before*** *the clock edge that data must be stable.*
- ***Hold Time ($t_{hold}$):*** *The time* ***after*** *the clock edge that data must remain stable.*

> **Important 4.5.** *If the setup or hold time is violated, the flip-flop may enter a* ***metastable*** *state where the output is stuck between 0 and 1 before eventually settling non-deterministically. This can cause system failure.*

**Definition 4.6.**

- ***Propagation Clock-to-Q Delay ($t_{pcq}$):*** *The* ***maximum*** *time after the clock edge until the output Q is stable.*
- ***Contamination Clock-to-Q Delay ($t_{ccq}$):*** *The* ***minimum*** *time after the clock edge until Q* ***starts*** *to change.*

**Theorem 4.7.** *To ensure a synchronous system operates correctly, two conditions must be met:*

- ***Setup Time Constraint:*** *The clock period $T_c$ must be long enough for the signal to propagate through the critical path:*

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

  *This determines the* ***maximum operating frequency*** *($f_{max} = 1/T_{c,min}$).*

- ***Hold Time Constraint:*** *The signal must not reach the next flip-flop too quickly (shortest path):*

$$t_{ccq} + t_{cd} > t_{hold}$$

  *If violated, data "shuffles" through multiple flip-flops in one cycle. This* ***cannot*** *be fixed by slowing the clock.*

## 4.3 Clock Skew and Performance

**Definition 4.8.** ***Clock skew*** *($t_{skew}$) is the time difference between the arrival of the clock signal at two different flip-flops. It is caused by the physical delay of the clock distribution network.*

**Important 4.9.** *Clock skew worsens both timing constraints:*

- ***Setup:*** $T_c \geq t_{pcq} + t_{pd} + t_{setup} + t_{skew}$ *(effectively increases required period).*
- ***Hold:*** $t_{ccq} + t_{cd} > t_{hold} + t_{skew}$ *(effectively increases required min delay).*

# 5 Von Neumann Model and ISA

**Definition 5.1.** *The **Von Neumann model**, proposed in 1946, defines a universal architecture for general-purpose computers consisting of five fundamental components:*

- ***Memory:*** *Stores both the program (instructions) and the data.*
- ***Processing Unit:*** *Performs arithmetic and logical operations. It contains the Arithmetic Logic Unit (ALU) and small temporary storage (registers).*
- ***Control Unit:*** *Orchestrates the execution of instructions. It contains the Program Counter (PC), which tracks the address of the next instruction, and the Instruction Register (IR), which holds the current instruction.*
- ***Input:*** *Devices to get information into the computer (e.g., keyboard).*
- ***Output:*** *Devices to get information out (e.g., monitor).*

**Important 5.2.** *Modern computing is based on two key properties of this model:*

- ***Stored Program:*** *Instructions are stored in a linear memory array alongside data. The hardware interprets a memory value as an instruction or data depending on which phase of the cycle it is in.*
- ***Sequential Processing:*** *Instructions are processed one at a time in sequence, unless a control flow instruction explicitly changes the Program Counter.*

**Remark 5.3.** *The process of executing a single instruction follows a specific sequence of phases:*

- *1.* ***FETCH:*** *Obtain instruction from memory into the IR and increment the PC.*
- *2.* ***DECODE:*** *Identify the opcode and determine the control signals needed.*
- *3.* ***EVALUATE ADDRESS:*** *Calculate the memory address of operands if needed (e.g., for loads).*
- *4.* ***FETCH OPERANDS:*** *Get the required source values from memory or registers.*
- *5.* ***EXECUTE:*** *Perform the actual operation in the ALU.*
- *6.* ***STORE RESULT:*** *Write the final value into the destination register or memory.*

**Definition 5.4.** ***Address Space:*** *The total number of uniquely identifiable memory locations. (LC-3: $2^{16}$; MIPS: $2^{32}$).*

**Definition 5.5.** ***Addressability:*** *The number of bits stored at each address (e.g., byte-addressable (8 bit) vs. word-addressable (32 bit)).*

**Definition 5.6.** ***Endianness:*** *The order of bytes in a word.* ***Big Endian*** *stores the Most Significant Byte (MSB) at the lowest address, while* ***Little Endian*** *stores the Least Significant Byte (LSB) at the lowest address.*

**Example 5.7.** *To read from memory:*

1. *Load the* ***Memory Address Register (MAR)*** *with the target address.*
2. *The memory hardware places the data from that address into the* ***Memory Data Register (MDR)****.*

*To write, both MAR (address) and MDR (data) are loaded before activating the write-enable signal.*

## 5.1 Instruction Set Architecture (ISA)

**Definition 5.8.** *The Instruction Set Architecture (ISA) serves as the "contract" or interface between the software (compiler/programmer) and the hardware (microarchitecture). It specifies:*

- ***Memory Organization:*** *Address space and addressability.*
- ***Register Set:*** *Number, size, and usage of registers (e.g., 8 GPRs in LC-3, 32 in MIPS).*
- ***Instruction Set:*** *Opcodes, data types, and addressing modes.*

**Important 5.9.** *Instructions are encoded as binary "words" containing:*

- ***Opcode:*** *Specifies the operation to perform (e.g., ADD, LD).*
- ***Operands:*** *Specify where to find the source data and where to put the result.*

**Lemma 5.10.** *The semantic gap refers to the distance between high-level language constructs and the ISA.*

- ***CISC (Complex Instruction Set Computing):*** *Small semantic gap; instructions do a lot of work (e.g., x86).*
- ***RISC (Reduced Instruction Set Computing):*** *Large semantic gap; simple instructions serve as primitives (e.g., MIPS, LC-3).*

**Definition 5.11.** *Mechanisms used to specify where an operand is located:*

- ***Immediate:*** *The operand is a constant within the instruction.*
- ***Register:*** *The operand is located in a general-purpose register.*
- ***PC-Relative:*** *The address is calculated as* $PC + offset$.
- ***Indirect:*** *The instruction points to a memory location that contains the actual address of the operand.*
- ***Base + Offset:*** *The address is calculated as* $Register + offset$.

## 5.2 Assembly

**Definition 5.12.** ***Machine code*** *is the binary representation (0s and 1s) processed by hardware.* ***Assembly language*** *is a human-readable representation using mnemonics (e.g., 'ADD') and labels for memory locations.*

**Example 5.13.** *High-level:* $a = b + c$

- ***LC-3 Assembly:*** *'ADD R0, R1, R2' (R0 is destination).*
- ***MIPS Assembly:*** *'add $s0, $s1, $s2' ($s0 is destination).*

**Important 5.14.** *In RISC (Reduced Instruction Set Computing) architectures, operands must be moved into registers before processing.*

- ***Load (LD/LDR/lw):*** *Reads data from memory into a register.*
- ***Store (ST/STR/sw):*** *Writes data from a register into memory.*

**Remark 5.15.** *The sequential execution order can be altered using:*

- ***Conditional Branches:*** *Executed only if a condition is met. LC-3 uses* ***Condition Codes*** *(N, Z, P) set by previous instructions. MIPS often compares two registers directly (e.g., 'beq s0,s1, Label').*
- ***Unconditional Jumps:*** *Always change the PC (e.g., 'JMP' in LC-3, 'j' in MIPS).*

# 6 Microarchitecture and Processor Design

## 6.1 Fundamentals

**Definition 6.1.** ***Microarchitecture*** *is the specific implementation of an Instruction Set Architecture (ISA) under given design constraints and goals. It describes how hardware structures, such as registers, ALUs, and buses, are organized to execute the instructions defined by the ISA.*

**Remark 6.2.** *While the ISA acts as the functional contract visible to the software, the microarchitecture is the internal logic that performs the work.*

**Important 6.3.** *A single ISA (such as x86, ARM, or MIPS)* ***can be implemented by many different microarchitectures****. This allows manufacturers to create different chips that run the same software but vary in performance, power consumption, or cost. The software remains unaware of whether the hardware uses a single-cycle, multi-cycle, or out-of-order execution strategy.*

**Remark 6.4.** *Microarchitects evaluate their designs based on several critical metrics:*

- ***Performance:*** *Usually measured by execution time or throughput.*
- ***Cost:*** *Often determined by the silicon die area and circuit complexity.*
- ***Power/Energy:*** *Critical for thermal management and battery-operated devices.*
- ***Reliability:*** *The ability of the system to operate correctly despite aging or environmental interference.*

## 6.2 Single-Cycle Microarchitecture

**Definition 6.5.** *In a single-cycle microarchitecture, every instruction is executed in exactly one clock cycle. This requires all phases of the instruction cycle, Fetch, Decode, Execute, Memory Access, and Write-back, to be completed within a single period of the global clock.*

**Important 6.6.** *The timing and performance of a single-cycle processor are characterized by:*

- ***CPI (Cycles Per Instruction):*** *The CPI is always 1.*
- ***Clock Cycle Time ($T_c$):*** *The clock period must be long enough to allow the*

***slowest*** *instruction in the ISA to complete.*

**Remark 6.7.** *In most RISC architectures, the lw (load word) instruction defines the critical path. It requires an instruction fetch, a register read, an ALU operation to calculate the address, a data memory read, and a register write. Because the clock must accommodate this long path, faster instructions like add or jump waste a significant portion of the clock cycle waiting for the next edge.*

## 6.3 Multi-Cycle Microarchitecture

**Definition 6.8.** *A multi-cycle microarchitecture breaks the execution of a single instruction into multiple, shorter clock cycles. Each instruction takes a variable number of cycles depending on its specific requirements (e.g., an add might take 3 cycles, while a lw takes 5).*

**Important 6.9.** *Multi-cycle designs allow for significant hardware reuse and require internal storage:*

- ***Functional Unit Reuse:*** *A single ALU can be used in different cycles of the same instruction (e.g., once to increment the PC and later to perform the arithmetic).*
- ***Microarchitectural Registers:*** *Since an instruction spans multiple cycles, intermediate results must be saved in registers that are not visible to the ISA, such as the Instruction Register (IR), Memory Data Register (MDR), and ALUOut.*

**Example 6.10.** *The phases of a multi-cycle instruction typically include:*

- ***Cycle 1 (Fetch):*** *Instruction is fetched and PC is incremented.*
- ***Cycle 2 (Decode):*** *Opcode is identified and registers are read.*
- ***Cycle 3 (Execute):*** *ALU performs the calculation or address computation.*
- ***Cycle 4 (Memory):*** *Data memory is accessed (only for loads/stores).*
- ***Cycle 5 (Write-back):*** *Result is written back to the register file.*

**Theorem 6.11.** *The execution time of a program is defined by the following equation:*

$$Time = Instruction\ Count \times Average\ CPI \times Clock\ Cycle\ Time$$

*While a multi-cycle design has a higher average CPI than a single-cycle design, it can achieve better performance because the clock cycle time ($T_c$) is much shorter,*

*as it only needs to cover the delay of the single longest stage rather than the entire instruction.*

**Remark 6.12.** *The* ***control unit*** *in a multi-cycle processor is implemented as a Finite State Machine (FSM). It must keep track of the current step of an instruction to ensure the correct control signals (like ALUSel or MemWrite) are asserted at the correct time.*

**Lemma 6.13.** *The primary* ***downside*** *of multi-cycle designs is the* ***"sequencing overhead"****. Each cycle transition adds a delay due to register setup times and clock-to-Q propagation. If instructions are broken into too many cycles, this overhead can eventually negate the performance benefits of a high clock frequency.*

# 7 Pipelining

## 7.1 Principles of Pipelining

**Definition 7.1.** ***Pipelining*** *is a microarchitecture implementation technique where* ***multiple instructions are overlapped*** *in execution. It functions like an assembly line, dividing the processing of an instruction into a sequence of stages so that a different instruction can be processed in each stage simultaneously.*

**Important 7.2.** *The core objective of pipelining is to increase* ***instruction throughput*** *(the number of instructions completed per unit of time). While it significantly improves throughput, it generally does not decrease the* ***latency*** *of a single instruction; in fact, the latency may increase slightly due to the overhead of pipeline registers.*

**Theorem 7.3.** *In an ideal scenario where all stages have equal delay and there are no dependencies, a pipeline with $k$ stages provides a $k$-fold speedup. However, in reality, the speedup is limited by:*

- ***Unbalanced stages:*** *The clock period is determined by the slowest stage.*
- ***Sequencing overhead:*** *Pipeline registers add setup time and clock-to-Q delays.*
- ***Hazards:*** *Dependencies between instructions that prevent seamless overlapping.*

## 7.2 The 5-Stage Instruction Pipeline

**Definition 7.4.** *A standard RISC pipeline (like MIPS or LC-3) typically consists of five stages:*

1. ***Instruction Fetch (IF):*** *Fetch the instruction from memory and update the PC.*
2. ***Instruction Decode (ID):*** *Decode the instruction and read operands from the register file.*
3. ***Execute (EX):*** *Perform an arithmetic/logic operation or calculate a memory address.*
4. ***Memory Access (MEM):*** *Read from or write to the data memory.*
5. ***Write-Back (WB):*** *Write the result back into the register file.*

**Important 7.5.** *To maintain the state of different instructions in different stages,* ***Pipeline Registers*** *(IF/ID, ID/EX, EX/MEM, MEM/WB) are placed between the stages. These registers carry the data and control signals forward through the pipeline every clock cycle.*

## 7.3 Pipeline Hazards and Dependencies

**Definition 7.6.** *A hazard is a situation that prevents the next instruction in the stream from executing in its designated clock cycle.*

- ***Structural Hazards:*** *Conflicts for hardware resources (e.g., using a single memory for both instructions and data).*
- ***Data Hazards:*** *An instruction depends on the result of a previous instruction that has not yet completed.*
- ***Control Hazards:*** *The next address to fetch is unknown because a branch or jump has not been resolved.*

**Definition 7.7.** *Data dependencies are primarily of three types:*

- ***Flow Dependence (RAW):*** *Instruction $j$ needs data produced by instruction $i$.*
- ***Anti-Dependence (WAR):*** *Instruction $j$ writes to a location that instruction $i$ reads.*
- ***Output Dependence (WAW):*** *Both instructions write to the same location.*

*In a standard 5-stage in-order pipeline, only RAW hazards require hardware intervention.*

## 7.4 Resolving Data Hazards

**Lemma 7.8.** ***Forwarding (Bypassing)*** *resolves many data hazards by routing the result of an instruction directly from the EX or MEM stage to the ALU input of a following instruction, bypassing the register file.*

**Important 7.9.** *Forwarding cannot resolve a* ***Load-Use Hazard*** *(where a load is immediately followed by an instruction using the loaded data). This requires a* ***Stall****. The hardware must:*

1. *Disable the PC and IF/ID register updates (holding instructions in place).*
2. *Flush the ID/EX register (inserting a "bubble" or NOP).*

## 7.5 Control Hazards and Branching

**Definition 7.10.** ***Control hazards*** *arise from the delay in determining the branch outcome. If the pipeline fetches the wrong instructions, it must* ***flush*** *those instructions once the branch is resolved.*

**Remark 7.11.** *Common strategies for handling branches include:*

- ***Static Prediction:*** *For example, "Always Predict Not Taken."*
- ***Dynamic Prediction:*** *Using a Branch History Table to predict based on past execution.*
- ***Early Branch Resolution:*** *Moving the branch comparison to the Decode stage to reduce the misprediction penalty.*

## 7.6 Advanced Pipeline Issues

**Important 7.12.** ***Precise Exceptions:*** *A pipelined processor must be able to stop execution on an exception-causing instruction while ensuring all prior instructions complete and no subsequent instructions have modified the architectural state. This is challenging because instructions finish at different times.*

**Lemma 7.13.** *In systems with multi-cycle functional units (e.g., a fast integer ALU and a slow divider), instructions can finish* ***out of order****. This "out-of-order completion" can break the sequential semantics of the ISA if an exception occurs in the middle of a stream.*

**Definition 7.14.** ***Fine-Grained Multithreading:*** *A technique where the processor switches to a different thread every clock cycle. This hides the latency of data and control hazards by ensuring that instructions currently in the pipeline are independent of one another.*